Multiple Agents Reacting to Environment

Brook Hamilton

hamilbro@oregonstate.edu

CS 475: Parallel Programming

Project 2

Professor Jesse Rosenzweig

October 11, 2026

Oregon State University - Cascades

Introduction:

This Project investigates parallelism for programming convenience rather than faster data computation. This investigation uses a month-by-month simulation of agents with individual threads that react to the state of the simulated world around them. The agents simulated in this project are present within a grain-growing operation which is affected by the temperature, amount of precipitation, and number of deer present in the area.

The number of deer, on the other hand, is affected by the amount of grain to eat as well as the number of monsters available to eat the deer, which was the agent I chose to add. This agent fits well within this simulation because it affects both the grain amount by getting trampled (-0.5 inches per month) and the deer population by them getting eaten (each monster can eat 1 deer per month). Its activity is also dependent on the temperature being over 50°F. The monster population is also determined by the number of deer available for consumption, which creates some interesting effects.

Using a C++ program, we wanted to investigate:

1. How the systems interact with each other
2. Using threads for convenience rather than for speed

Machine Details:

Machine: Lenovo LOQ AMD Ryzen 7 7435HS
Architecture: X86_64
CPU(s): 16
Thread(s) per core: 2
Core(s) per socket: 8

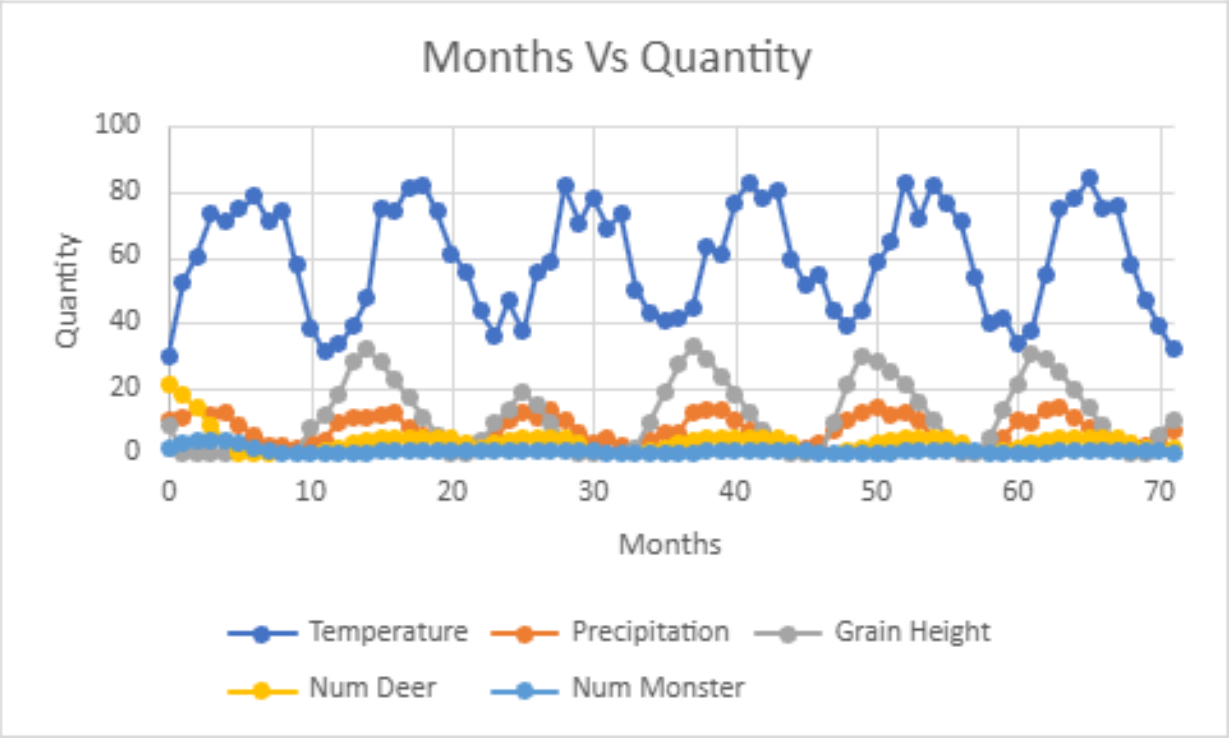
Results:

Year	Month	Time (Months)	Temperature	Precipitation	Grain Height	Num Deer	Num Monster
2024	0	0	30	10	8.914555	21	2
2024	1	1	52.661621	10.820172	0	18	3
2024	2	2	60.485603	13.989315	0	14	4

2024	3	3	73.409332	11.58576	0	9	4
2024	4	4	70.846588	12.315559	0	4	4
2024	5	5	74.874008	8.768795	0	0	3
2024	6	6	78.866455	5.962569	0	0	2
2024	7	7	71.437828	2.810963	0	0	1
2024	8	8	74.22097	2.869225	0	0	0
2024	9	9	57.537853	2.073633	0.295487	0	0
2024	10	10	38.689919	3.185234	7.709213	0	0
2024	11	11	31.007492	4.418634	11.623911	1	0
2025	0	12	33.426117	9.769622	18.40905	2	0
2025	1	13	38.991447	10.846418	28.202812	3	0
2025	2	14	47.41943	11.23079	32.018852	4	0
2025	3	15	75.154877	11.668583	28.018902	5	1
2025	4	16	74.400787	12.59909	22.518984	5	1
2025	5	17	81.571312	7.737041	17.018984	5	1
2025	6	18	82.069557	5.544234	11.518984	5	1
2025	7	19	74.013794	4.648459	6.019069	5	1
2025	8	20	61.026711	2.289876	0.598666	5	1
2025	9	21	55.358521	2.2841	0	3	1
2025	10	22	43.862442	4.323476	3.989431	2	1
2025	11	23	36.347775	4.856921	9.55018	3	1
2026	0	24	46.835976	10.229021	13.5665	4	1
2026	1	25	37.252968	13.039948	19.212017	5	1
2026	2	26	55.343529	11.139778	14.836897	5	1
2026	3	27	59.02066	13.448462	9.622868	5	1

2026	4	28	81.946793	10.638212	4.122868	5	1
2026	5	29	70.601936	6.633007	0	3	1
2026	6	30	78.472549	3.699469	0	1	1
2026	7	31	68.907738	4.639896	0	0	0
2026	8	32	73.220535	2.608124	0.000112	0	0
2026	9	33	50.156937	1.363486	2.02878	0	0
2026	10	34	43.362007	3.798353	9.32438	1	0
2026	11	35	40.93219	6.117982	18.556339	2	0
2027	0	36	41.313614	6.710036	27.141068	3	0
2027	1	37	44.610619	12.96998	33.023903	4	0
2027	2	38	63.439816	13.679364	29.066982	5	1
2027	3	39	60.862251	13.749693	23.701235	5	1
2027	4	40	76.941711	10.658835	18.201248	5	1
2027	5	41	83.075737	7.216811	12.701248	5	1
2027	6	42	78.120605	6.967386	7.201254	5	1
2027	7	43	80.726158	2.078708	1.701254	5	1
2027	8	44	59.755741	2.777935	0	3	1
2027	9	45	51.830822	1.951124	0	1	1
2027	10	46	54.987232	3.111919	0	0	0
2027	11	47	43.82756	6.881793	9.404349	0	0
2028	0	48	39.472683	10.248794	21.363625	1	0
2028	1	49	43.826599	12.501708	30.100231	2	0
2028	2	50	58.507988	14.439444	28.420807	3	0
2028	3	51	64.826195	11.658855	25.445383	4	0
2028	4	52	83.147186	12.923153	21.445383	5	1

2028	5	53	72.271713	10.077164	15.945744	5	1
2028	6	54	82.140121	5.174899	10.445744	5	1
2028	7	55	76.534065	1.881597	4.945753	5	1
2028	8	56	70.896416	0.434276	0	3	1
2028	9	57	53.76429	0.108871	0	1	1
2028	10	58	39.608532	1.862297	4.678973	0	0
2028	11	59	41.810356	5.113092	13.824949	1	0
2029	0	60	34.073627	10.18013	21.268166	2	0
2029	1	61	37.921288	9.746942	30.753326	3	0
2029	2	62	54.732502	13.837456	28.935293	4	0
2029	3	63	74.871414	14.535571	24.935345	5	1
2029	4	64	77.831039	10.775393	19.435352	5	1
2029	5	65	84.313927	8.027569	13.935352	5	1
2029	6	66	75.201721	4.376132	8.435389	5	1
2029	7	67	75.831909	1.73501	2.935405	5	1
2029	8	68	58.224171	2.133039	0	3	1
2029	9	69	47.333118	2.378326	0.420665	2	1
2029	10	70	39.139908	3.737644	5.967976	1	1
2029	11	71	32.172077	7.247503	10.495766	2	0



Conclusion:

Based on the simulation I ran and the data provided in both the table and the graph that shows the full spread of data over 6 years, it is very obvious that there is a strong correlation between precipitation and the well-being of all other agents present within the simulation. As there is more precipitation in the early months of the year, the grain grows taller; as the grain grows taller, the deer and monster quantities rise. The presence of the monsters has a very interesting effect on the overall well-being of both the deer and grain. Although the main driver for grain health is without a doubt shown to be temperature, with precipitation also being a large contributing factor.

The ever-present threat of the monsters and consequential effect on deer population creates a particularly frustrating effect on both species' populations; the deer are never able to grow their population over about 5, and the monsters hunt too much and too fast to grow their own population without overhunting the deer. This is the result of the population only being allowed to grow by 1 per month and the monsters only eating 1 per month. So the population of deer is held at 5 until the grain runs out. Thus, populations are held to a very low amount, with the monsters staying more or less constant throughout the entirety of the 6 years. After trying multiple different starting populations for both the deer and monsters, it was found that this

hunting issue happens no matter the number of each species, as they are both dependent on the other.

In conclusion, this kind of simulation was very interesting to tweak around and mess with to try to further influence the results and responses by each of the agents lacrosse their individual threads, and gave great insight into another use case of parallel programming. Having multiple computations happening across multiple threads deepened the understanding and importance of barriers for both private and shared variables for the processes to exit and complete in an accurate and effective manner.